Tasca S4.01. BigQuery Avançat & Analytics Engineering

Descripció

Com a Analytics Engineer, la teva responsabilitat no és només lliurar dades, sinó assegurar l'escalabilitat i l'eficiència de la plataforma.

En aquest sprint final, redissenyaràs l'arquitectura de les dades per optimitzar costos (**Partitioning**), introduiràs lògica analítica avançada (**Window Functions**) i modelaràs estructures complexes (**Arrays**) per tancar el cicle amb l'automatització.

⚠️ **Important**

- » **Control de Costos**: A BigQuery, cada consulta costa diners. Abans d'executar res, fixa't sempre en el missatge del validador ("Dry Run") a dalt a la dreta.
- » **Ubicació de les Dades**: Assegura't que el teu Dataset estigui a la regió EU (Multiple regions) o les connexions fallaran.

Limitacions del Sandbox

La zona de proves de BigQuery (Sandbox) està subjecta als següents límits:

- » S'apliquen tots els límits i quotes habituals de BigQuery.
- » Tens 10 GB d'emmagatzematge actiu i 1 TB de dades de consulta processades cada mes sense cap cost.
- » ⚠️ **Caducitat**: Totes les taules, vistes i particions caduquen de forma automàtica després de **60 dies**.
- » No admet funcions avançades com: Transmissió de dades (Streaming), DML (UPDATE/DELETE massius) o el Servei de transferència de dades.

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari: El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'**Alemanya** necessita revisar urgentment les transaccions del dia **12 de març de 2022**.

🧩 **Tasca:**

- » 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

👀 *Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).*

La consulta estima procesar 11,86 MB si se ejecuta. Esto puede resultar ineficiente desde el punto de vista de los costes, ya que, al no estar la tabla particionada, es necesario leer una gran parte de la tabla para obtener los datos correspondientes a un solo día.



Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari: Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un *Full Table Scan* (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

🧩 **Tasca:**

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada sprint3_silver.transactions_recent a partir d e la taula sprint3_silver.transactions_clean. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar SELECT * EXCEPT(timestamp). Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions TIMESTAMP_SUB(), CURRENT_TIMESTAMP(), i la co mbinació CAST(RAND() * 50 AS INT64) per generar els dies de desfasament.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva sprint3_gold.fact_transactions_optimized a partir de les dades recents que acabes de generar a transactions_recent. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

- » **Particionament (Partitioning)**: Divideix la taula per la data del camp DATE(timestamp). Això permetrà que les consultes que filtrin per dia (WHERE date = ...) només llegeixin la partició necessària.
- » **Clustering**: Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINs) amb la dimensió d'empreses.

👉 **Entrega:**

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (CREATE OR REPLACE TABLE...) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula fact_transactions_optimized on es pugui comprovar que està particionada i agrupada en clústers correcta ment.

Para crear la nueva tabla, utilicé SELECT * EXCEPT(timestamp), tal como se solicita en el ejercicio, para mantener todas las columnas originales excepto timestamp. También utilicé las funciones CURRENT_TIMESTAMP(), RAND(), CAST() y TIMESTAMP_SUB() para generar nuevas fechas aleatorias, desde el momento actual hasta un máximo de 49 días atrás. De esta manera, todas las transacciones

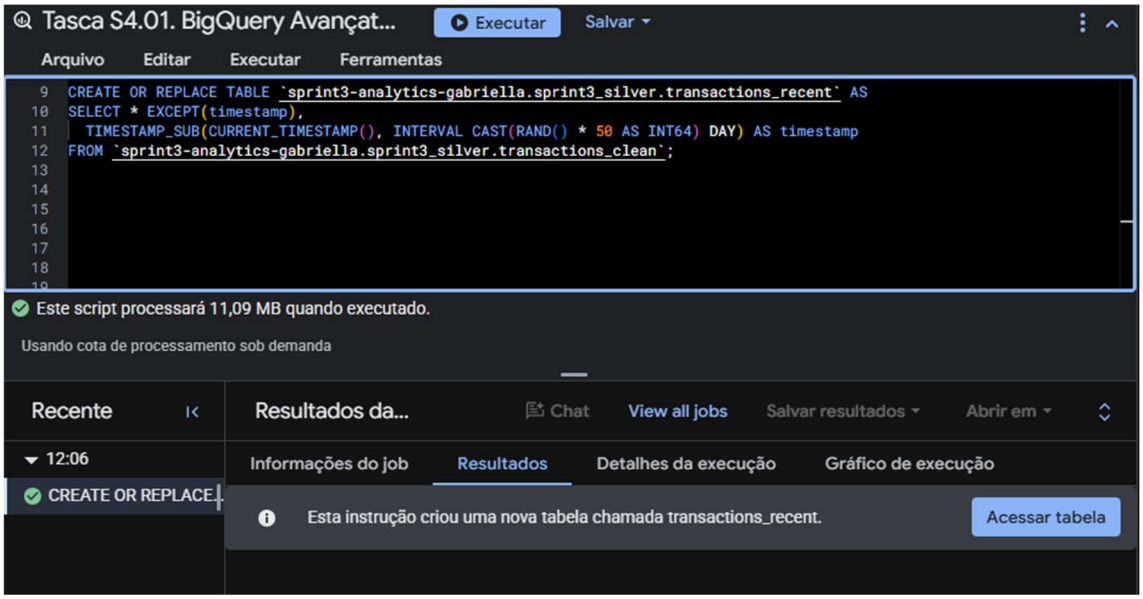
tienen un timestamp reciente, lo que permitirá trabajar con el particionamiento en el siguiente paso.

CREATE OR REPLACE TABLE `sprint3-analytics-gabriella.sprint3_silver.transactions_recent` AS

SELECT * EXCEPT(timestamp),

TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp

FROM `sprint3-analytics-gabriella.sprint3_silver.transactions_clean`;



Iniciamos la creación de la tabla con `CREATE OR REPLACE TABLE`. A continuación, definimos el particionamiento por fecha mediante `PARTITION BY DATE(timestamp)` y configuramos el clustering por identificador de empresa con CLUSTER BY business_id. De esta manera, optimizamos el almacenamiento para facilitar las consultas que filtran por fecha o por empresa.

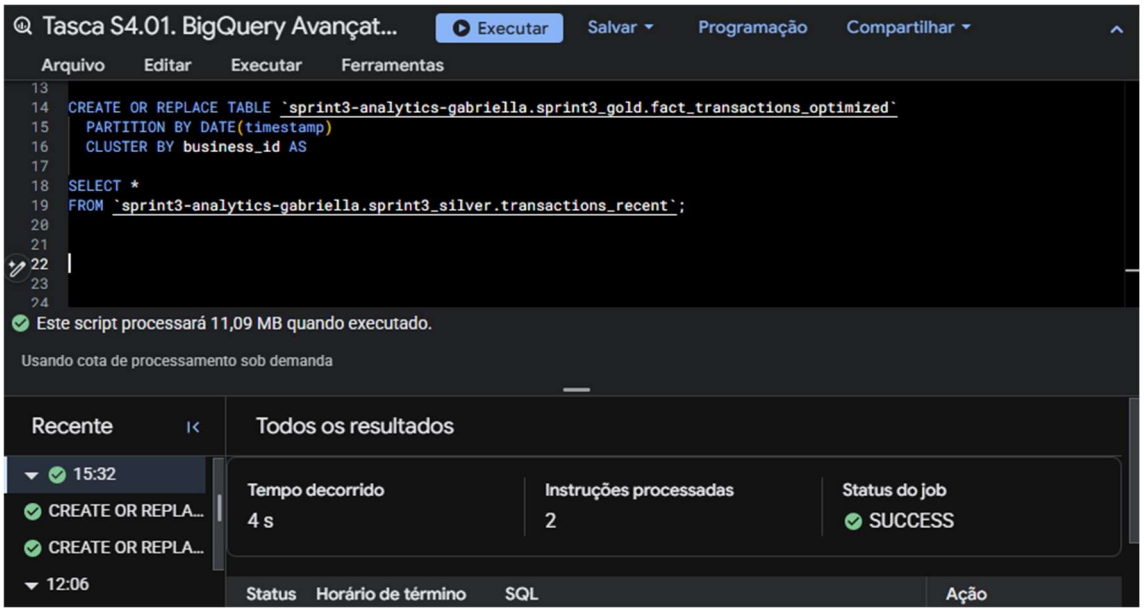
CREATE OR REPLACE TABLE `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized`

PARTITION BY DATE(timestamp)

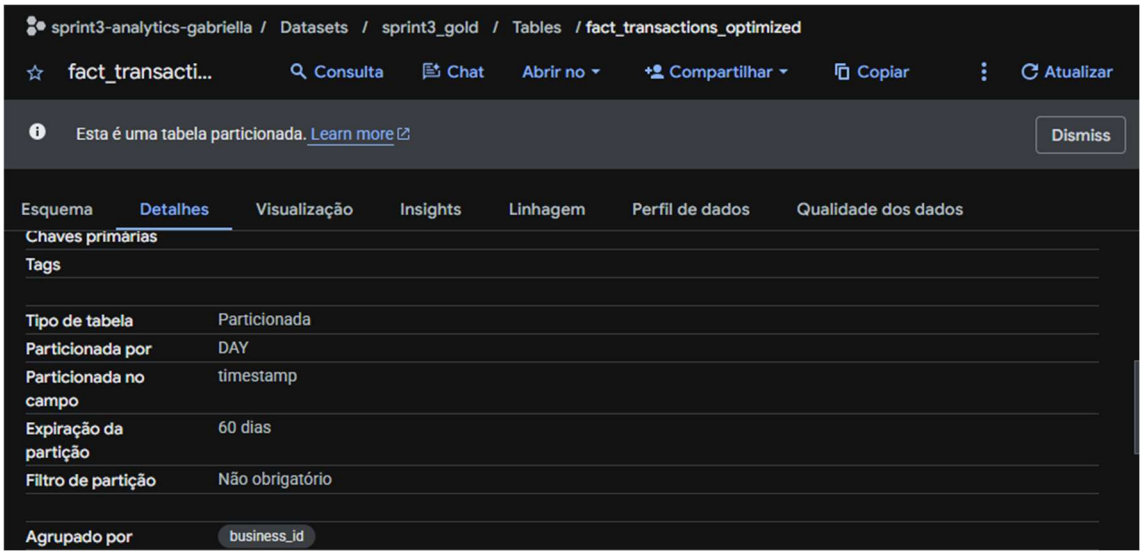
CLUSTER BY business_id AS

SELECT *

FROM `sprint3-analytics-gabriella.sprint3_silver.transactions_recent`;



La tabla se ha creado correctamente. En sus detalles podemos comprobar que está particionada por la fecha del campo timestamp y que utiliza clustering por business_id para optimizar las consultas que filtran por empresa.



Exercici 3: La Prova del Cotó (Benchmark)

Escenari: Ja has creat la taula sprint3_gold.fact_transactions_optimized amb les estratègies de Particionament i Clustering.
Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

🧩 **Tasca:**

L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional**.

Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i filtri les dades dels últims 30 dies.

⚠️ **NO executis les consultes encara.** En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

- » **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula sprint3_silver.transactions_recent. Observa el validador i fes una captura de pantalla dels bytes que processaria.
- » **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada sprint3_gold.fact_transactions_optimized. Observa com canvia el validador i fes la segona captura.

👉 **Entrega:**

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el **% d'estalvi** que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

Para comparar ambas tablas, utilizamos la misma consulta SQL, seleccionando todas las columnas mediante SELECT * y filtrando las transacciones de los últimos 30 días. Los resultados muestran una reducción de los bytes procesados de 11,85 MB a 7,22 MB, lo que representa un ahorro aproximado del **39,07 %**.

Esta mejora resulta especialmente relevante cuando trabajamos con conjuntos de datos de mayor tamaño, ya que el particionamiento puede reducir la cantidad de datos leídos y, por tanto, los costes de las consultas.

```
SELECT *

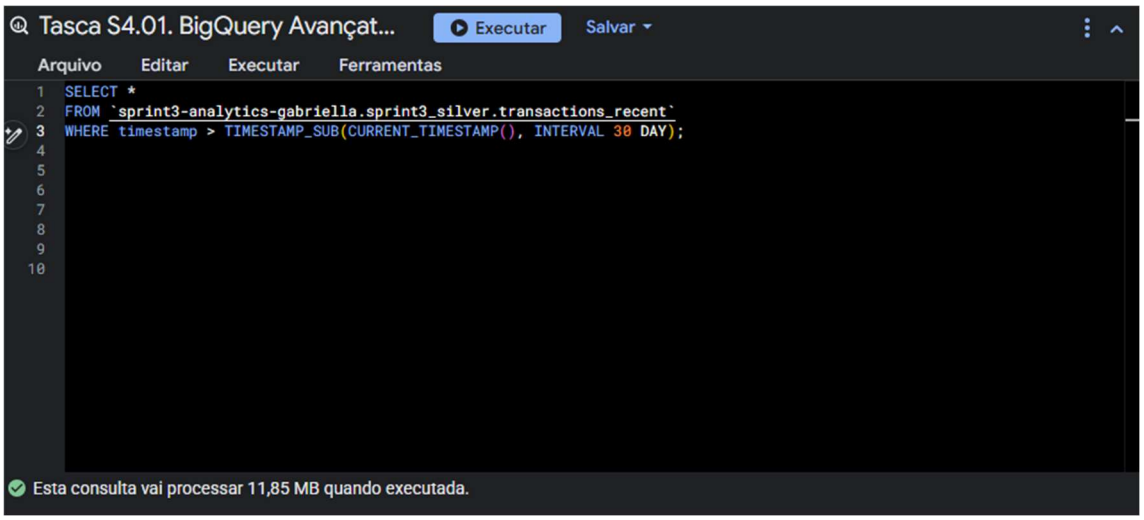
FROM `sprint3-analytics-gabriella.sprint3_silver.transactions_recent`

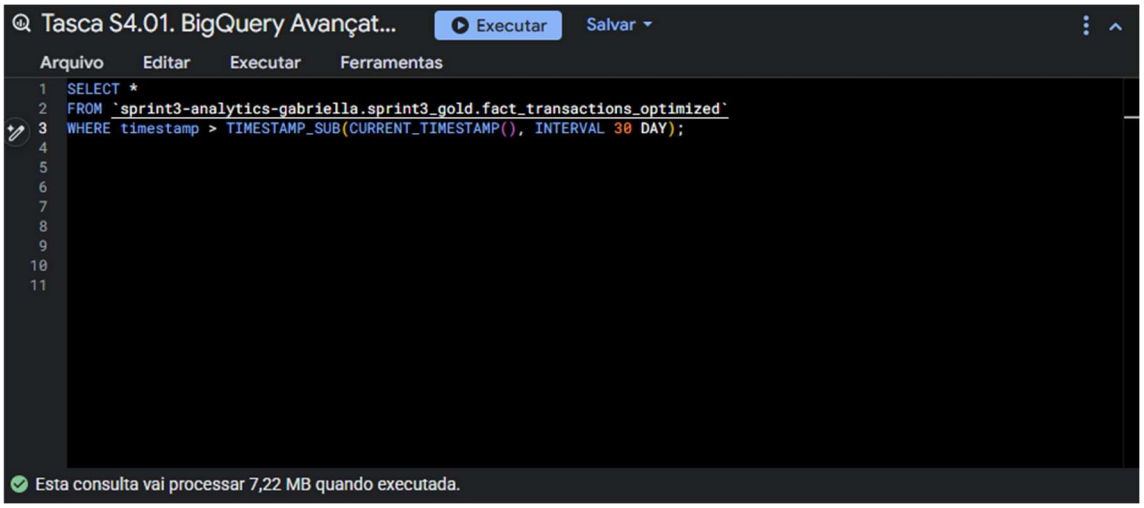
WHERE timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```

SELECT *

FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized`

WHERE timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);





Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari: El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema: Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució: Utilitzar una **Vista Materialitzada**. Aquesta tecnologia guarda el *resultat* del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

🧩 **Tasca:**

- » Crea una **Vista Materialitzada** anomenada sprint3_gold.mv_daily_sales que mostri les **Vendes Totals per Dia**.

👉 **Entrega:**

- » Codi per Crear la **Vista Materialitzada**.
- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

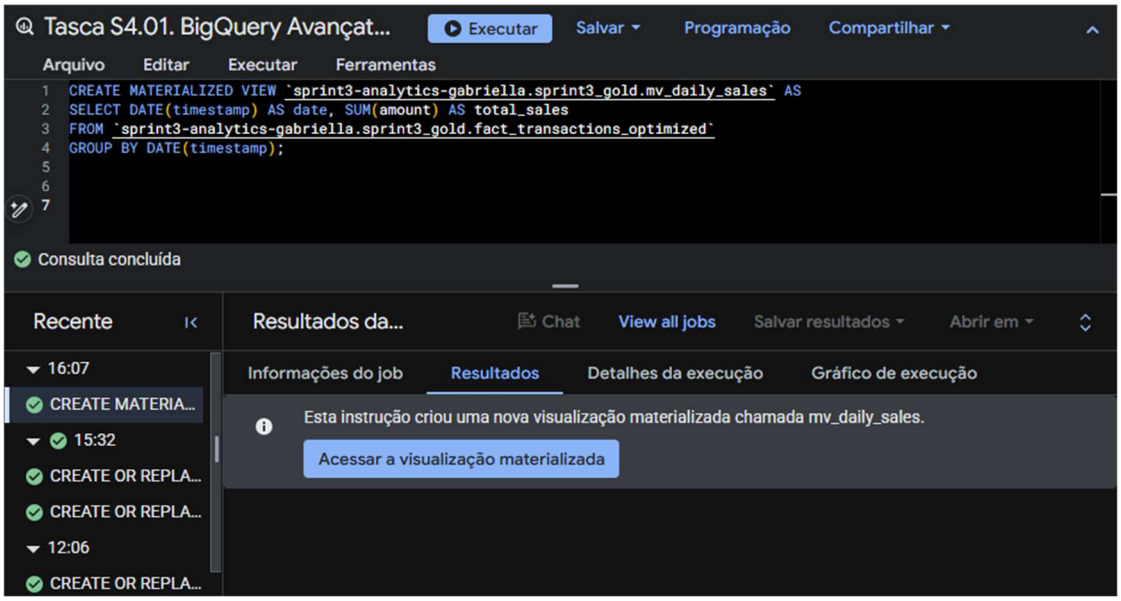
La vista materializada se creó mediante CREATE MATERIALIZED VIEW, utilizando la tabla particionada fact_transactions_optimized que habíamos creado anteriormente. A partir de esta tabla, agrupamos las transacciones por fecha mediante GROUP BY y calculamos las ventas totales de cada día con SUM(amount), seleccionando la fecha y el importe total de las ventas.

CREATE MATERIALIZED VIEW `sprint3-analytics-gabriella.sprint3_gold.mv_daily_sales` AS

SELECT DATE(timestamp) AS date, SUM(amount) AS total_sales

FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized`

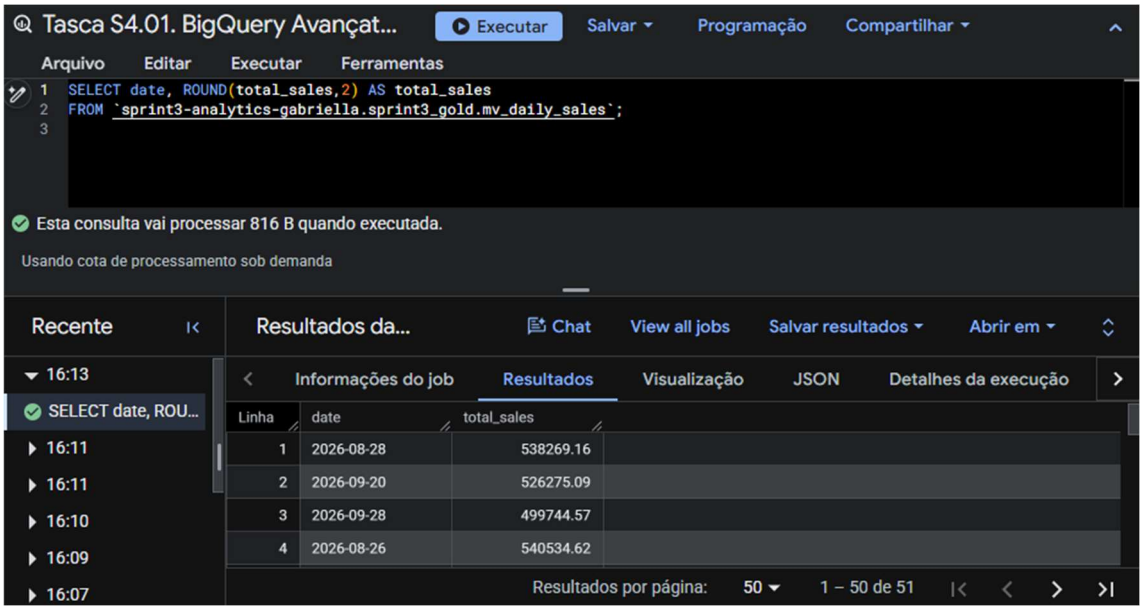
GROUP BY DATE(timestamp);



A continuación, consultamos la vista materializada mediante SELECT para visualizar los resultados. Como no fue posible aplicar la función ROUND() durante la creación de la vista, la utilizamos posteriormente en la consulta de visualización para redondear los importes a dos decimales.

SELECT date, ROUND(total_sales,2) AS total_sales

FROM `sprint3-analytics-gabriella.sprint3_gold.mv_daily_sales`;



Nivell 2: SQL Analític Avançat

Escenari: La direcció comença a fer preguntes que no es resolen amb un simple GROUP BY. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

Escenari: L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

🧩 **Tasca:**

- **» 1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi**:
    - La Despesa Total (SUM).
    - La Quantitat de Transaccions (COUNT).

- El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
- La Compra Màxima (MAX).
- Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

- **» Creua la CTE amb users_combined per obtenir les dades personals.**

Requisits de Sortida:

- Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenat per total_gastat descendent.

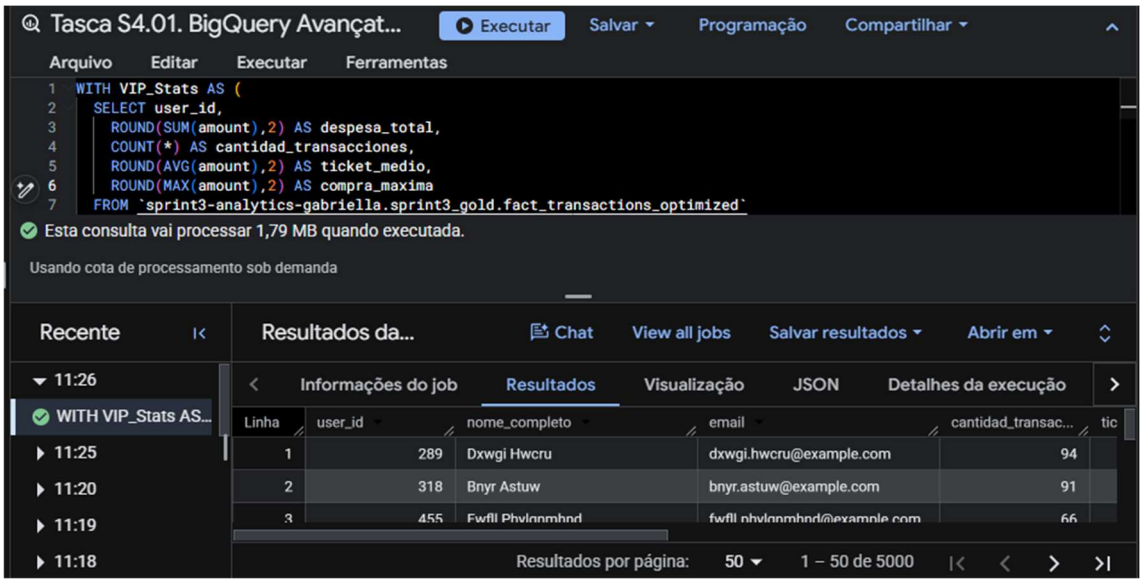
Para crear la CTE, utilizamos WITH VIP_Stats AS y agrupamos las transacciones por user_id. A continuación, calculamos las principales métricas de comportamiento de compra: el número de transacciones mediante COUNT(), el gasto total mediante SUM(), el importe medio mediante AVG(), y el importe máximo de una compra mediante MAX(), redondeado a dos decimales con ROUND().

Utilizamos HAVING para filtrar los usuarios cuyo gasto total supera los 500 € y, posteriormente, consultamos la CTE mediante un JOIN con la tabla users_combined para obtener los datos personales de cada cliente, como su nombre completo y su correo electrónico.

Por último, ordenamos los resultados por gasto total en orden descendente, de modo que los clientes VIP que más gastan aparezcan primero.

```
WITH VIP_Stats AS (
  SELECT user_id,
    ROUND(SUM(amount),2) AS despesa_total,
    COUNT(*) AS cantidad_transacciones,
    ROUND(AVG(amount),2) AS ticket_medio,
    ROUND(MAX(amount),2) AS compra_maxima
  FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized`
  GROUP BY user_id
  HAVING despesa_total > 500
)
```

```
SELECT
 v.user_id,
 CONCAT(u.name, ' ', u.surname) AS nome_completo,
 u.email,
 v.cantidad_transacciones,
 v.ticket_medio,
 v.compra_maxima,
 v.despesa_total
FROM VIP_Stats v
JOIN `sprint3-analytics-gabriella.sprint3_silver.users_combined` u ON v.user_id = u.user_id
ORDER BY v.despesa_total DESC;
```



Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari: La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

📈 **Estratègia d'Optimització:** Per a aquest càlcul NO has d'anar a la taula de fets original ni recalcular sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada sprint3_gold.mv_daily_sales que vas crear a l'exercici anterior. Això

garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

🧩 **Tasca:**

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

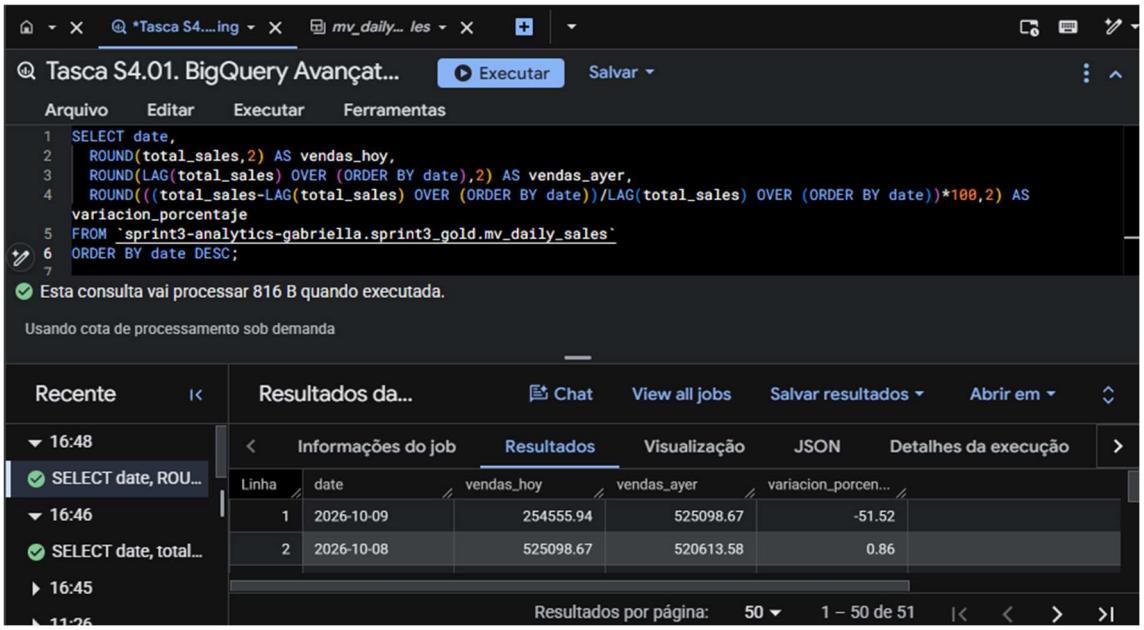
- » 1.**Data**: La data de venda.
- » 2.**Vendes_Avui**: El total venut aquell dia.
- » 3.**Vendes_Ahir**: El total venut el dia anterior (usant funcions de finestra).
- » 4.**Diff_Percentual**: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

💡 **Pista Tècnica:**

Utilitza la funció LAG(camp) OVER (ORDER BY data) per llegir el valor de la fila anterior sense necessitat de fer JOINs complexos.

Para crear el informe, utilizamos LAG(total_sales) OVER (ORDER BY date) para obtener el importe total de las ventas del día anterior. De esta manera, podemos comparar las ventas de dos días consecutivos y calcular la variación porcentual, identificando posibles aumentos o disminuciones en el rendimiento diario. Además, utilizamos ROUND() para redondear los importes y el porcentaje a dos decimales, y ordenamos los resultados por fecha en orden descendente para mostrar primero los días más recientes.

```
SELECT date, ROUND(total_sales,2) AS vendas_hoy,
 ROUND(LAG(total_sales) OVER (ORDER BY date),2) AS vendas_ayer,
 ROUND(((total_sales-LAG(total_sales) OVER (ORDER BY date))/LAG(total_sales) OVER (ORDER BY date))*100,2) AS variacion_porcentaje
FROM `sprint3-analytics-gabriella.sprint3_gold.mv_daily_sales`
ORDER BY date DESC;
```



Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari: El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

🧩 **Tasca:**

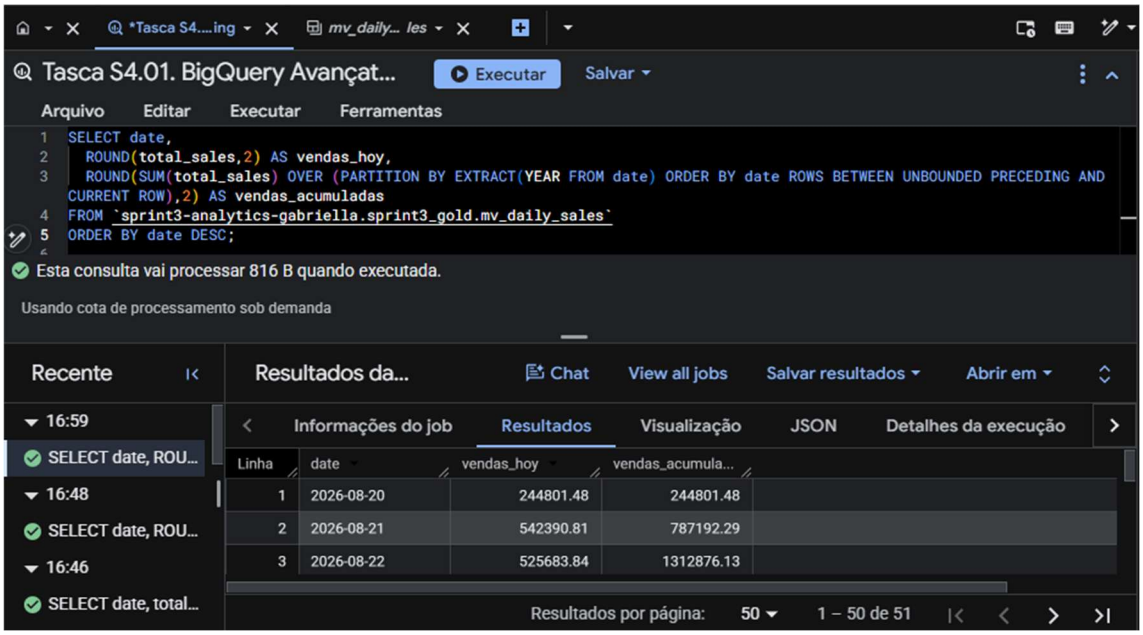
Utilitzant la vista materialitzada mv_daily_sales (per no recalcular agregacions base), genera un informe amb tres columnes:

- » 1.**Data**.
- » 2.**Vendes del Dia**: Arrodonit a 2 decimals.
- » 3.**Vendes Acumulades YTD**: Una suma progressiva de les vendes que ha de reiniciar-se cada **1 de Gener**. El resultat final ha d'estar arrodonit a 2 decimals.

💡 **Pista Tècnica:** Per aconseguir el reinici anual, necessites dividir la finestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Siguiendo la pista técnica del ejercicio, utilizamos SUM() OVER() junto con PARTITION BY EXTRACT(YEAR FROM date) para calcular las ventas acumuladas. De esta manera, sumamos progresivamente las ventas de cada día y reiniciamos el acumulado al comienzo de cada año. Finalmente, utilizamos ROUND() para redondear los importes a dos decimales.

```
SELECT date,
  ROUND(total_sales,2) AS vendas_hoy,
  ROUND(SUM(total_sales) OVER (PARTITION BY EXTRACT(YEAR FROM date) ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2) AS vendas_acumuladas
FROM `sprint3-analytics-gabriella.sprint3_gold.mv_daily_sales`
ORDER BY EXTRACT(YEAR FROM date) DESC;
```



Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari: L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

🧩 **Tasca:**

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1.**Dades d'usuari (user_id, nom_complet, email)**.
- » 2.**La data i l'import exacte de la 3a compra**.
- » 3.**Mitjana de les 3 primeres**: La mitjana de despesa de les seves transaccions 1, 2 i 3.

💡 **Pista Tècnica:**

- **» Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres**.
- **» Has d'utilitzar la clàusula QUALIFY per filtrar**.
- **» Repte d'Optimització**: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

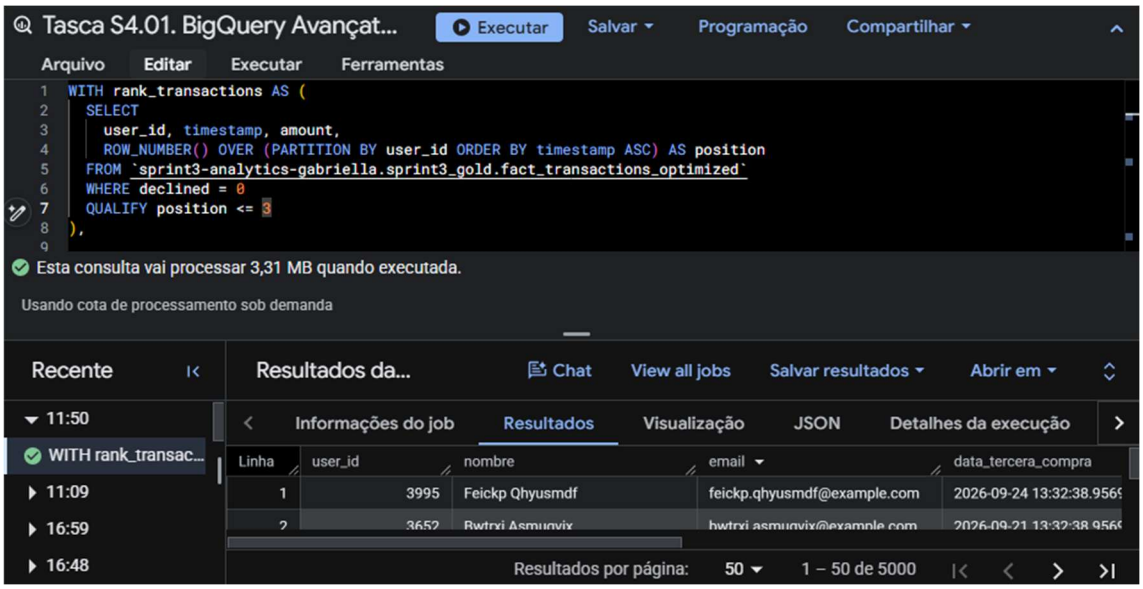
Para obtener el resultado deseado y optimizar el uso de recursos, primero creamos las CTEs rank_transactions, que ordena las compras cronológicamente y asigna una posición a cada transacción, y first_three, que garantiza que analizamos únicamente a los clientes con al menos tres transacciones y calcula el importe medio de sus tres primeras compras. Por último, realizamos la consulta final y los JOIN necesarios para incorporar los datos personales de los usuarios y presentar el informe solicitado por el equipo de Marketing, incluyendo la fecha y el importe de la tercera compra, así como la media de las tres primeras transacciones.

```
WITH rank_transactions AS (
  SELECT
    user_id, timestamp, amount,
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp ASC) AS position
  FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized`
  WHERE declined = 0
  QUALIFY position <= 3
),

first_three AS (
  SELECT
    user_id, AVG(amount) AS media_primeras_tres
  FROM rank_transactions
  GROUP BY user_id
```

```
  HAVING COUNT(*) = 3
)

SELECT
  u.user_id,
  CONCAT(u.name, ' ', u.surname) AS nombre,
  u.email,
  t.timestamp AS data_tercera_compra,
  ROUND(t.amount,2) AS total_tercera_compra,
  ROUND(f.media_primeras_tres, 2) AS media_primeras_tres
FROM rank_transactions t
JOIN first_three f ON t.user_id = f.user_id
JOIN `sprint3-analytics-gabriella.sprint3_silver.users_combined` u ON t.user_id = u.user_id
WHERE t.position = 3
ORDER BY t.timestamp DESC;
```



Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari: Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

Escenari: Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

🧩 **Tasca:**

Crea la taula dim_transactions_flat desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

⚙️ **Passos Tècnics:**

- » 1. Utilitza CROSS JOIN UNNEST(product_ids) per transformar l'Array en files individuals.
- » 2. Realitza un JOIN amb la taula products per obtenir el nom (name) i preu (price) de cada ítem.

📌 *Nota: Ves amb compte amb els tipus de dades en fer el JOIN (Array de INT64 vs Product_ID STRING).*

- » 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

✅ **Resultat Esperat (Visualització):**

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

👉 **Entrega:**

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

Para crear la nueva tabla, utilicé CROSS JOIN UNNEST() para descomponer el array de productos y obtener una fila por cada elemento. A continuación, realicé un JOIN con la tabla products_clean para recuperar el nombre y el precio de cada producto. También utilicé CAST() para convertir los identificadores al mismo tipo de dato y garantizar que la comparación entre ambas tablas fuera compatible.

Finalmente, comprobé la creación de la tabla dim_transactions_flat, revisando su esquema y la vista previa de los datos en BigQuery.

```
CREATE OR REPLACE TABLE `sprint3-analytics-gabriella.sprint3_gold.dim_transactions_flat` AS

SELECT t.transaction_id,

 t.timestamp,

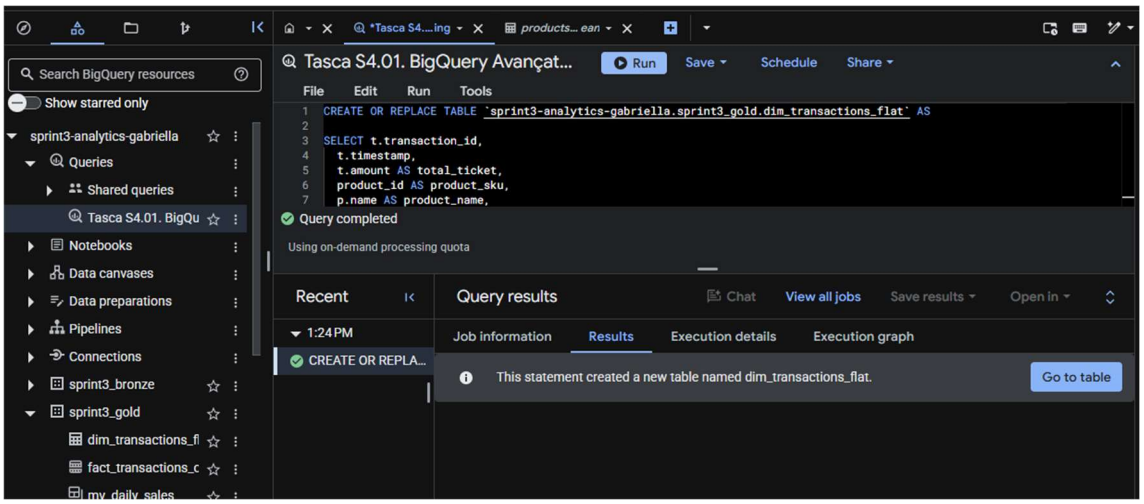
 t.amount AS total_ticket,

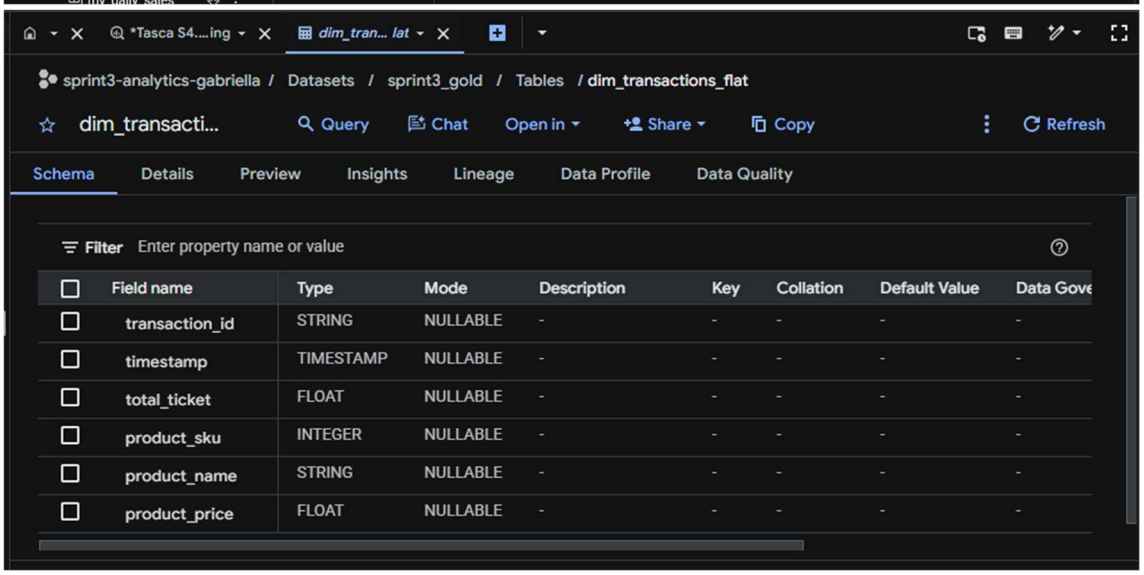
 product_id AS product_sku,

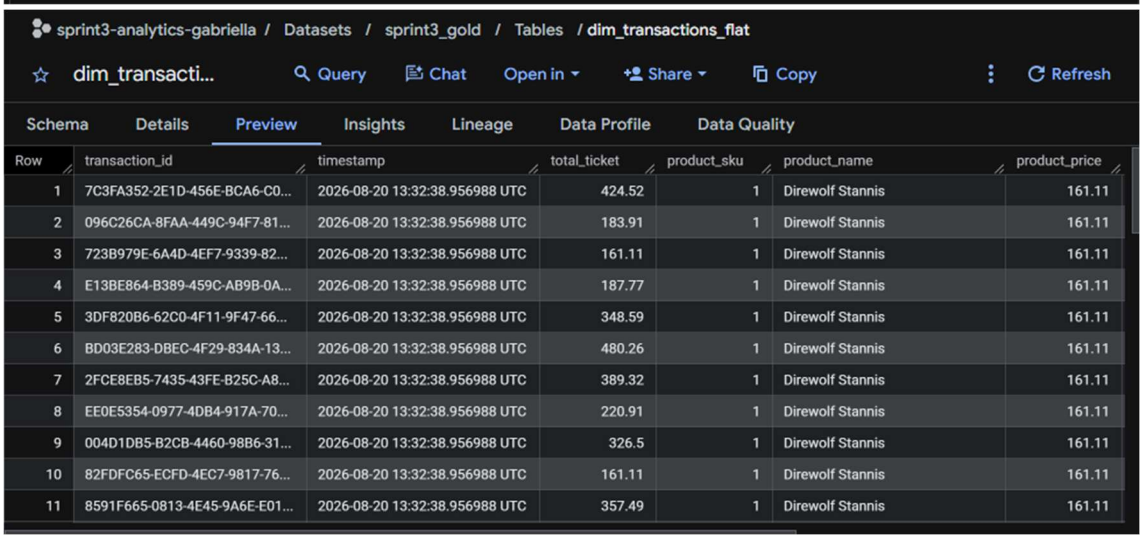
 p.name AS product_name,

 p.price AS product_price
```

```
FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized` t
CROSS JOIN UNNEST(product_ids) AS product_id
JOIN `sprint3-analytics-gabriella.sprint3_silver.products_clean` p
ON CAST(product_id AS STRING) = CAST(p.product_id AS STRING);
```







Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari: Ara que has preparat la infraestructura i disposes de la taula dim_transactions_flat (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

🧩 **Tasca:**

Genera el **Top 5 de productes** més venuts en la història de la companyia.

🛠️ **Mètode:**

Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).

- » En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.

👉 **Entrega:**

- » El codi SQL que genera el rànquing.
- » Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

Para generar el ranking, agrupé los productos por SKU y nombre mediante GROUP BY y utilicé COUNT(*) para calcular la cantidad de unidades vendidas. Después, ordené los resultados de mayor a menor con ORDER BY y apliqué LIMIT 5 para obtener el Top 5 de los productos más vendidos.

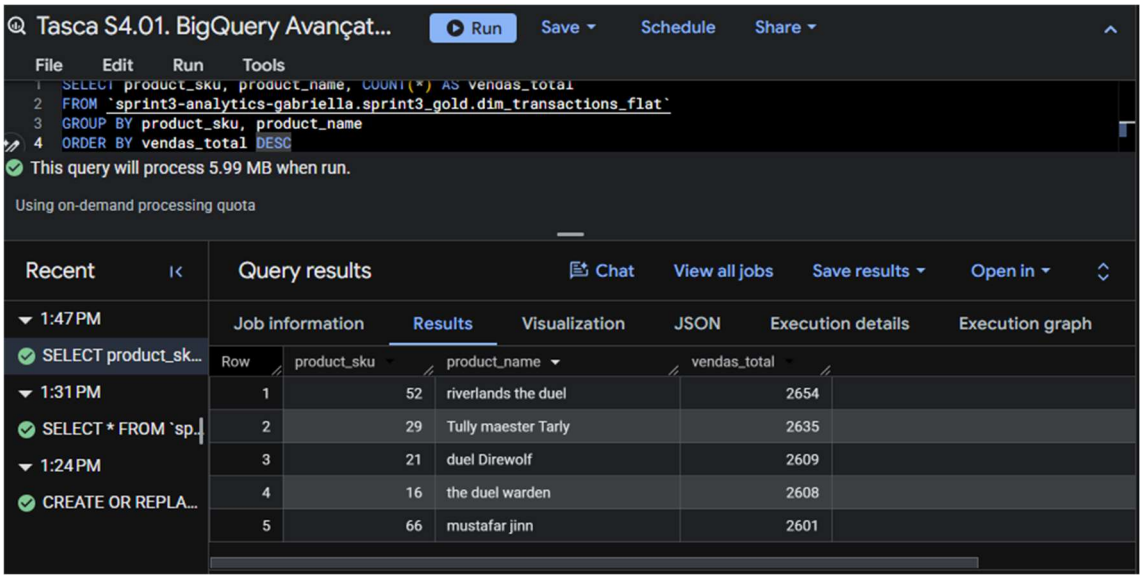
```
SELECT product_sku, product_name, COUNT(*) AS vendas_total

FROM `sprint3-analytics-gabriella.sprint3_gold.dim_transactions_flat`

GROUP BY product_sku, product_name

ORDER BY vendas_total DESC

LIMIT 5;
```



Exercici 3: Automatització del Pipeline i Visualització

Escenari: L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- **» 1. Lògica Centralitzada**: Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul preu * 1,21 directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- **» 2. Frescor de Dades**: El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

🧩 **Tasca:**

Has d'evolucionar la teva taula dim_transactions_flat per incloure el càlcul d'impostos i automatitzar la seva regeneració.

⚙️ **Passos Tècnics:**

- **» 1. User Defined Functions (UDF)**: Crea una funció SQL persistent anomenada calculate_tax(amount) que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
- **» 2. Integració i Orquestració**:
    - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció calculate_tax i generi una columna nova: product_price_tax_inc.

    - Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.

- » **3. Visualització (BI)**: Connecta Looker Studio a la teva taula dim_transactions_flat i crea un Dashboard: "Monitor de Rendiment de Vendes".

👉 **Entrega:**

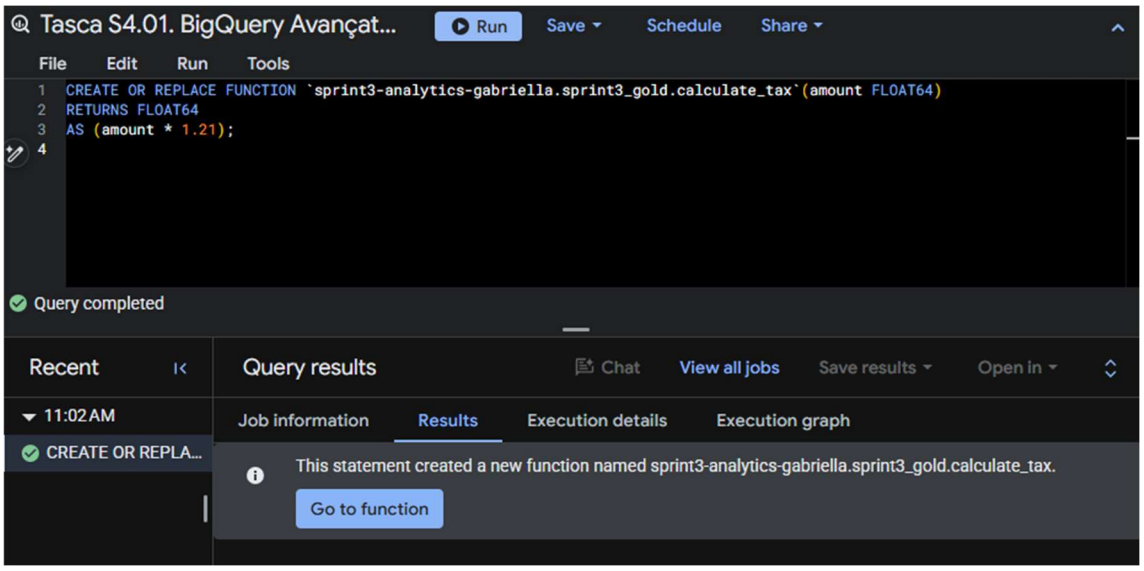
- » El codi SQL de la UDF.
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la Scheduled Query està programada.
- » Un enllaç del gràfic a Looker Studio.

Primero, creamos una función SQL reutilizable para calcular el precio final de cada producto con el impuesto incluido.

CREATE OR REPLACE FUNCTION `sprint3-analytics-gabriella.sprint3_gold.calculate_tax`(amount FLOAT64)

RETURNS FLOAT64

AS (amount * 1.21);



A continuación, actualizamos la tabla dim_transactions_flat, creada en el ejercicio 1, incorporando una nueva columna llamada product_price_tax_inc, que utiliza la función creada anteriormente para calcular el precio de cada producto con el IVA del 21 % incluido. También añadimos la columna last_update para registrar la fecha y hora de la última actualización de la tabla.

```
CREATE OR REPLACE TABLE `sprint3-analytics-gabriella.sprint3_gold.dim_transactions_flat` AS

SELECT t.transaction_id,

 t.timestamp,

 t.amount AS total_ticket,

 product_id AS product_sku,

 p.name AS product_name,

 p.price AS product_price,

 `sprint3-analytics-gabriella.sprint3_gold.calculate_tax`(p.price) AS product_price_tax_inc,

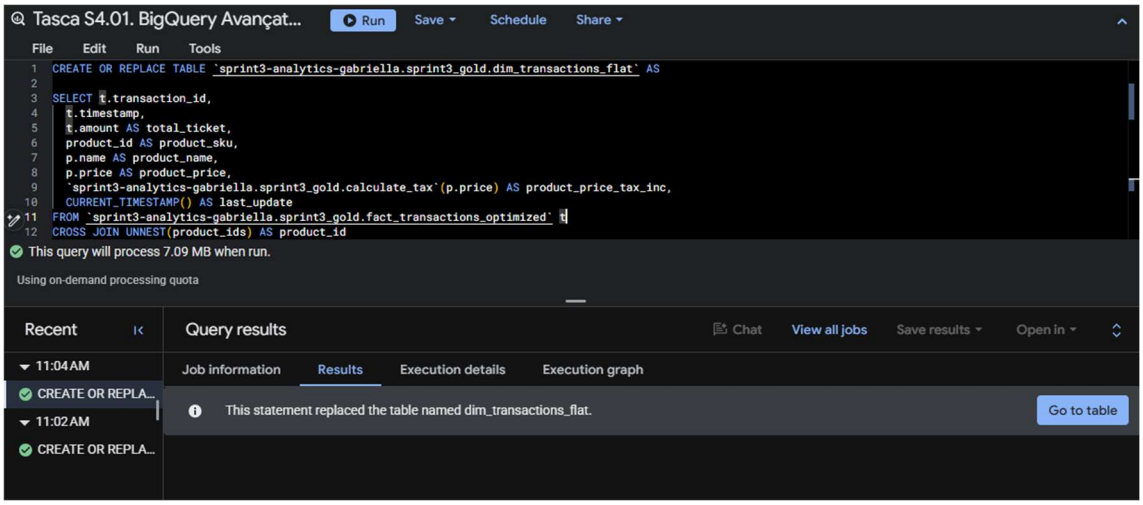
 CURRENT_TIMESTAMP() AS last_update

FROM `sprint3-analytics-gabriella.sprint3_gold.fact_transactions_optimized` t

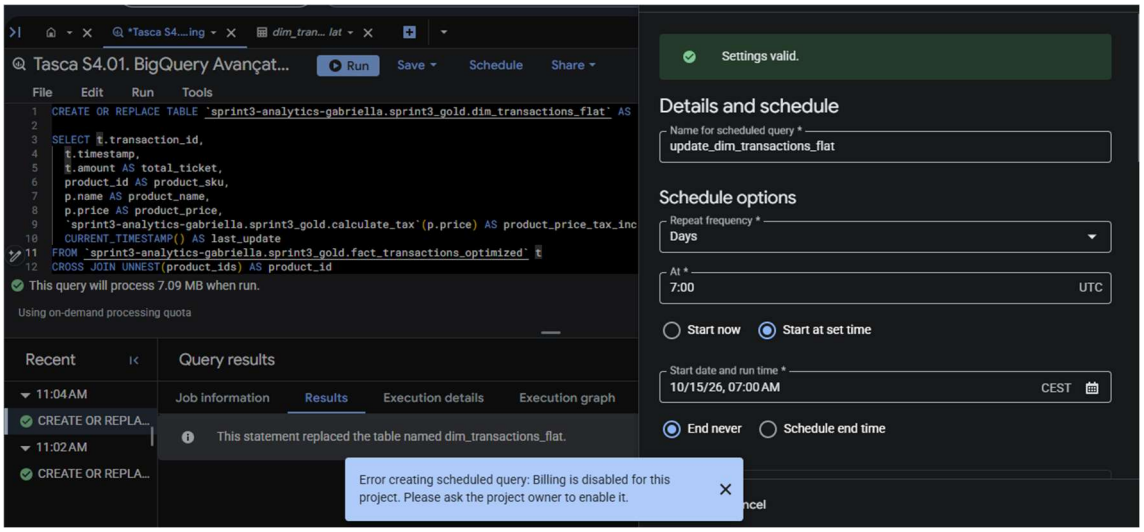
CROSS JOIN UNNEST(product_ids) AS product_id

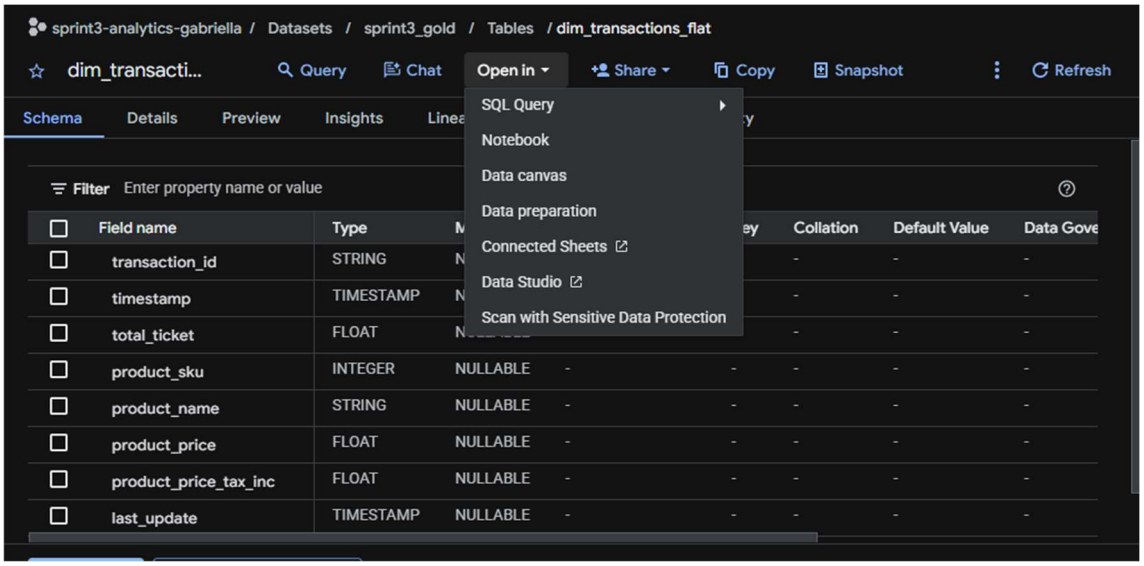
JOIN `sprint3-analytics-gabriella.sprint3_silver.products_clean` p

ON CAST(product_id AS STRING) = CAST(p.product_id AS STRING);
```



Por último, intenté programar la ejecución automática de la consulta todos los días a las 07:00, pero no fue posible completar la configuración porque el proyecto de Google Cloud no tenía una cuenta de facturación (*billing account*) asociada.





Enllaç del gràfic a Looker Studio:

https://datastudio.google.com/s/g5V_W7JH8y4

📌 **Proposta de Dashboard:**

"Monitor de Rendiment de Vendes"

Configura una pàgina amb 3 seccions clau:

- **» 1. Els "Big Numbers" (KPIs Generals):**

A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.

- **KPI 1: Ingressos Totals (Amb IVA)**
  - **Mètrica**: SUM(product_price_tax_inc)
  - *Per què*: Mostra el volum real de diners que entra.
- **Transaccions Úniques (Comandes)**
  - **Mètrica**: COUNT_DISTINCT(transaction_id)
- **Transaccions Úniques (Comandes)**
  - **Camp Calculat**: Crea un camp a Looker: SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)

- **» 2. Evolució Temporal (Gràfic de Sèries Temporals)**

Sota els KPIs, un gràfic de línia que ocupi tot l'ample.

- **Dimensió**: timestamp (Desglossat per Data).
- **Mètrica**: SUM(product_price_tax_inc).
- **Línia de Referència (Opcional)**: Afegeix una mitjana constant o una línia de tendència.
- **Valor**: Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

- **» El "Top Productes" Enriquit (Taula amb Mapa de Calor)**

Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.

- **Dimensió**: product_name.
- **Mètrica 1**: COUNT(*) → Renombra-ho a "Unitats Venudes".
- **Mètrica 2**: SUM(product_price_tax_inc) → Renombra-ho a "Ingressos Totals".
- **Mètrica 3**: SUM(product_price_tax_inc) - SUM(product_unit_price) → Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).
- **Estil**: A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.